SecureChat

End-to-End Encrypted Messaging System

Data Security Project Report

Course: Data Security · AAST · Spring 2026

Submitted by:

Abduallah Ragab — 221007888
Ali Ibrahim Katary — 221017764
Mazen Wael — 221007781
Mohammed Mostafa — 221017636
Mustafa Ahmed — 221017920

Submitted to:

Dr. Amina El Hawary
Eng. Jomana Ahmed

1. What We Built

SecureChat is a web-based end-to-end encrypted (E2EE) 1:1 messaging application. Two users can register, become contacts, and exchange messages — but the server never sees plaintext. Even a full database breach reveals no readable messages.

Core claim: The server stores only ciphertext. All encryption and decryption happens entirely in the browser using the native Web Crypto API.

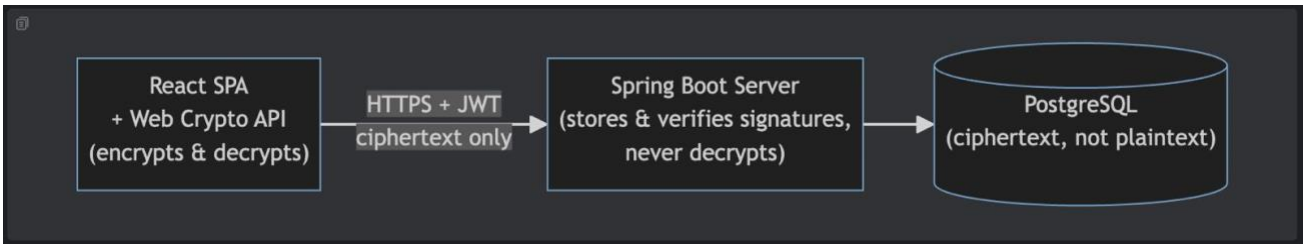

Figure 1 — High-level system flow: React SPA → Spring Boot Server → PostgreSQL

1.1 Technology Stack

Layer	Technology
Frontend	React 19 + TypeScript + Vite
Client-side Cryptography	Web Crypto API (browser-native, no external libraries)
Backend	Spring Boot 3.5 (Java 17)
Database	PostgreSQL + Flyway migrations
Authentication	JWT (HS-256)
Backend Testing	JUnit 5 + Mockito
Frontend Testing	Vitest + Playwright

---

2. System Architecture

The architecture is divided into three trust zones. The browser is the only trusted zone where plaintext and decryption keys exist. The server verifies signatures and stores encrypted blobs but has no ability to decrypt anything. The database contains only ciphertext, hashes, and public keys.

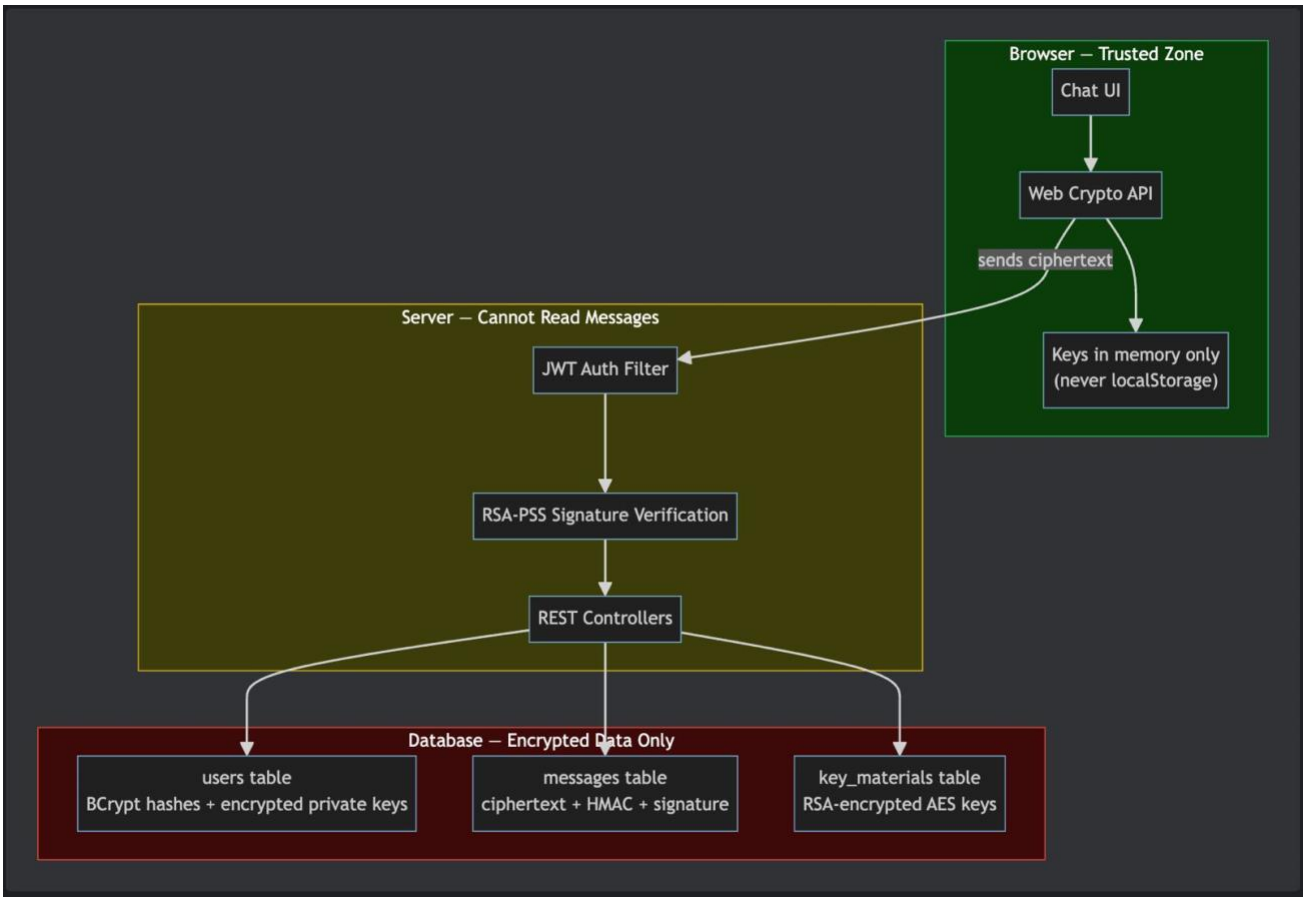

Figure 2 — Full system architecture: Browser (Trusted Zone) → Server (Cannot Read Messages) → Database (Encrypted Data Only)

Zone	Role & Contents
Browser — Trusted Zone	Chat UI, Web Crypto API, RSA & AES keys held in memory only (never localStorage)
Server — Cannot Read Messages	JWT Auth Filter → RSA-PSS Signature Verification → REST Controllers
Database — Encrypted Data Only	users table (BCrypt + encrypted private keys), messages table (ciphertext + HMAC + signature), key_materials table (RSA-encrypted AES keys)

---

3. Cryptographic Algorithms

Six cryptographic primitives are used across the system. No external crypto libraries are used — the browser uses the native Web Crypto API and the server uses Java's built-in SunJCE provider.

Purpose	Algorithm	Where	Standard
Password storage	BCrypt (cost 12)	Server	OpenBSD / NIST
Private key protection	PBKDF2-SHA256 (600K iter) + AES-256-GCM	Browser	NIST SP 800-132
Key exchange	RSA-OAEP SHA-256 (2048-bit)	Browser	PKCS #1 v2.2
Key exchange auth	RSA-PSS SHA-256 (salt = 32)	Browser signs, Server verifies	PKCS #1 v2.1
Message encryption	AES-256-GCM (256-bit key, 12-byte IV)	Browser	NIST FIPS 197
Message integrity	HMAC-SHA256	Browser computes, Server verifies	RFC 2104
Message authentication	RSA-PSS SHA-256 (2048-bit)	Browser signs, Server verifies	PKCS #1 v2.1
Replay protection	Timestamp ±5 min window	Server	—
Session tokens	JWT HS-256 (24h expiry)	Server	RFC 7519

---

4. Registration — Key Generation & Protection

When a user registers, the browser generates an RSA key pair and encrypts the private key with a password-derived key before sending it to the server. The server never receives the private key in plaintext.

Step	Action
1	User enters password in the browser
2	Browser generates RSA-2048 key pair (Web Crypto API)
3	PBKDF2(password, random salt, 600K iterations) → AES-256 key
4	AES-256-GCM encrypt(RSA private key) → encrypted blob + IV
5	POST /auth/register { email, password, publicKeyPem, encryptedPrivateKey, iv }
6	Server: BCrypt hash(password), stores password_hash + publicKey + encryptedPrivateKey

7	Server responds with JWT token — private key and AES key remain in browser memory only

Security note: The server receives the private key already encrypted. Without the user's password, it is computationally irretrievable (PBKDF2 with 600,000 SHA-256 iterations). BCrypt (cost 12) protects the password hash in the database.

---

5. Key Exchange — Delivering the Conversation Key

When Alice starts a conversation with Bob, she generates a random AES-256 key and encrypts it with Bob's RSA public key. This is a hybrid encryption scheme — RSA for key transport, AES for message encryption — the standard approach used in TLS, PGP, and S/MIME.

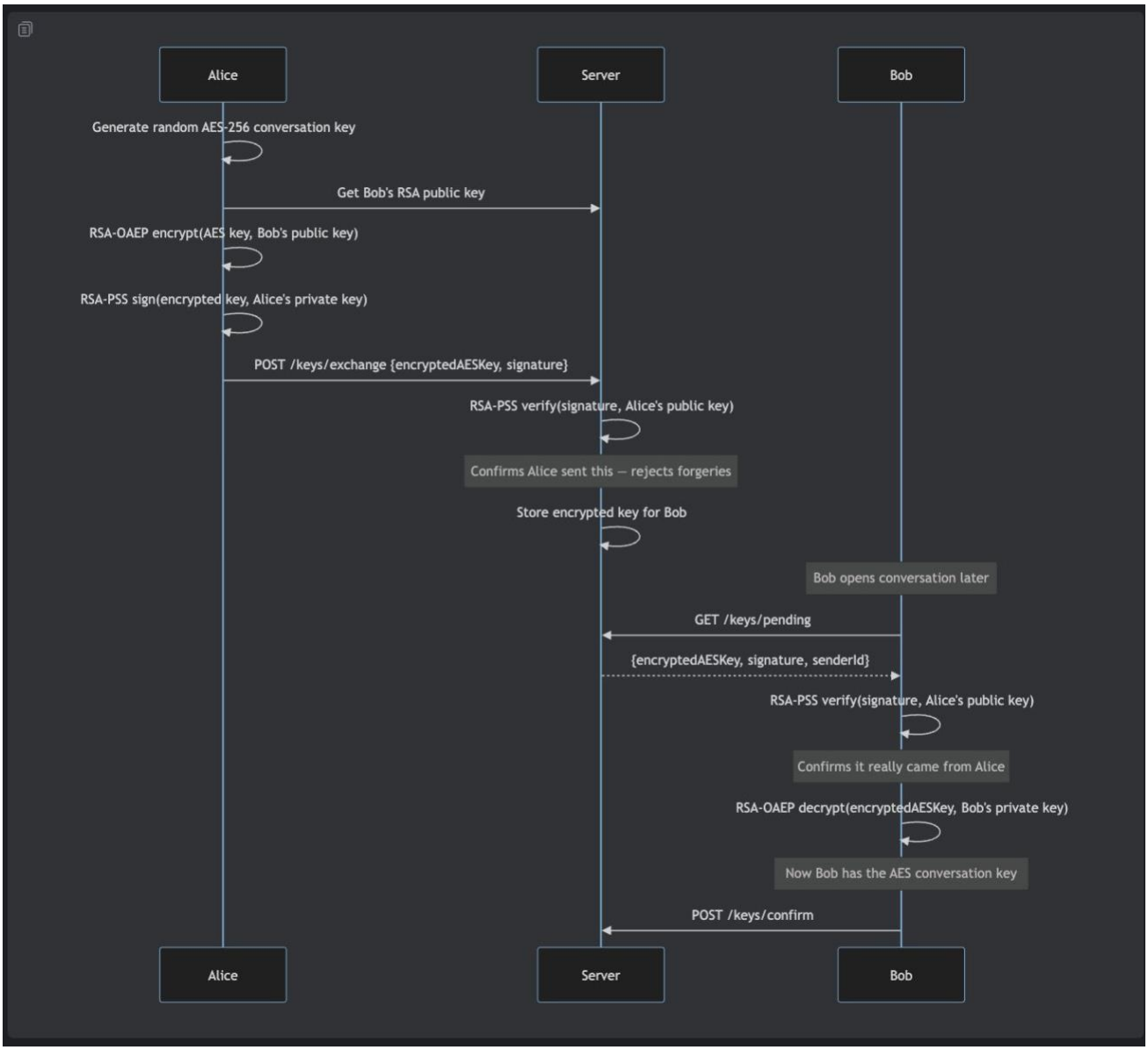

Figure 3 — Key Exchange sequence diagram: Alice generates AES-256 key, encrypts with Bob's public key, signs with her private key; Bob verifies and decrypts

Server role: The server verifies Alice's RSA-PSS signature to confirm the sender's identity and reject forgeries, but it cannot read the AES key — it only stores the RSA-encrypted blob for Bob to retrieve.

---

6. Sending & Receiving Messages

Every message goes through three cryptographic layers before transmission: AES-256-GCM encryption, HMAC-SHA256 integrity verification, and RSA-PSS digital signature. The server performs authentication checks without ever decrypting the content.

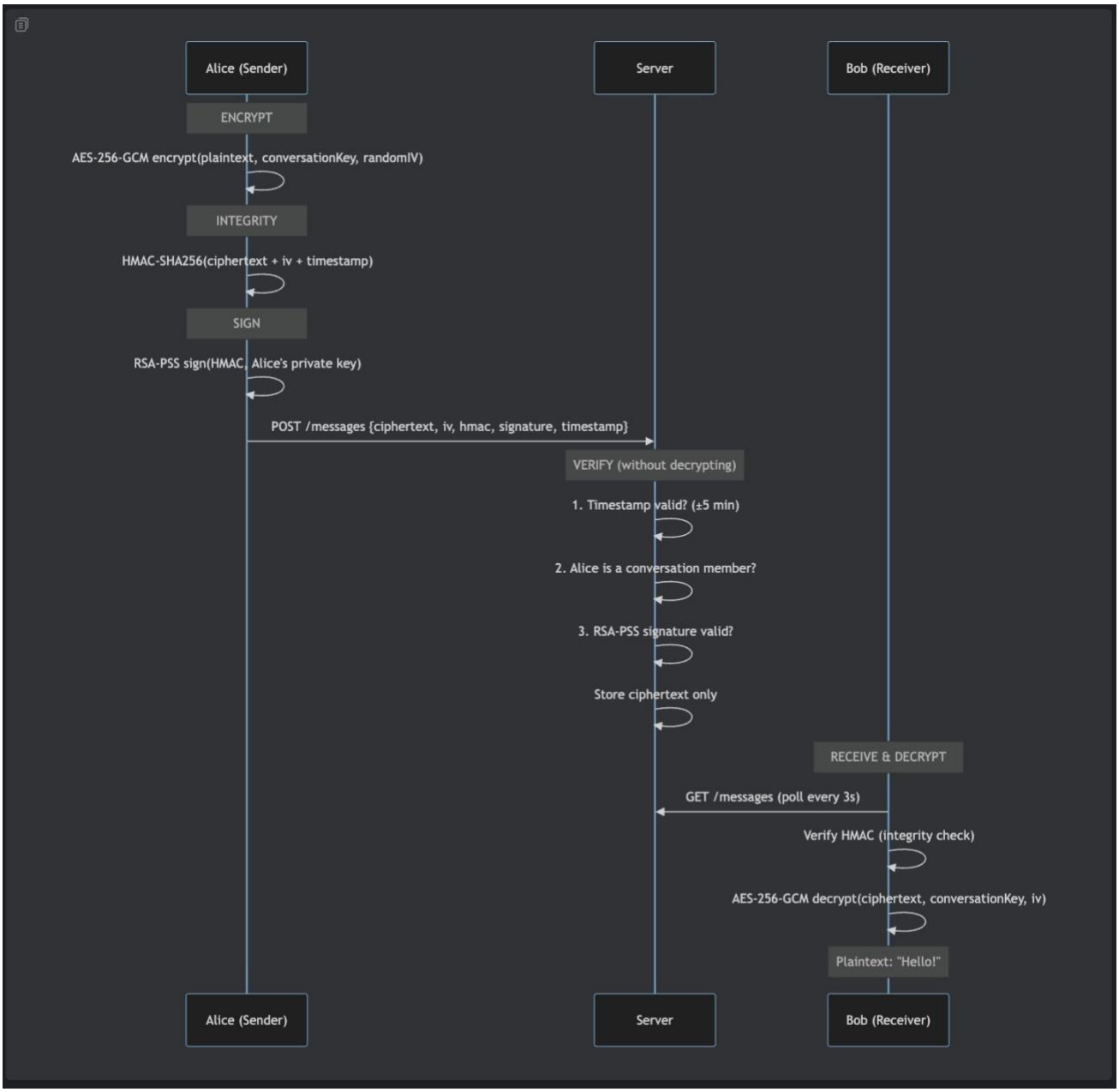

Figure 4 — Message send/receive sequence: Encrypt → HMAC → Sign → Server verifies → Bob decrypts

6.1 What Gets Stored Per Message

Column	Value	Readable by server?

ciphertext	AES-256-GCM encrypted blob	No
iv	12-byte random IV	No (IVs are public but useless alone)
hmac	HMAC-SHA256 hash	No (for integrity, not decryption)
signature	RSA-PSS signature	No (for authentication, not decryption)
timestamp	Unix milliseconds	Yes (metadata only)
sender_id	User ID	Yes (metadata only)

---

7. Database Schema

The database stores no plaintext. Every sensitive field is encrypted before storage. The schema below shows all six tables and their relationships.

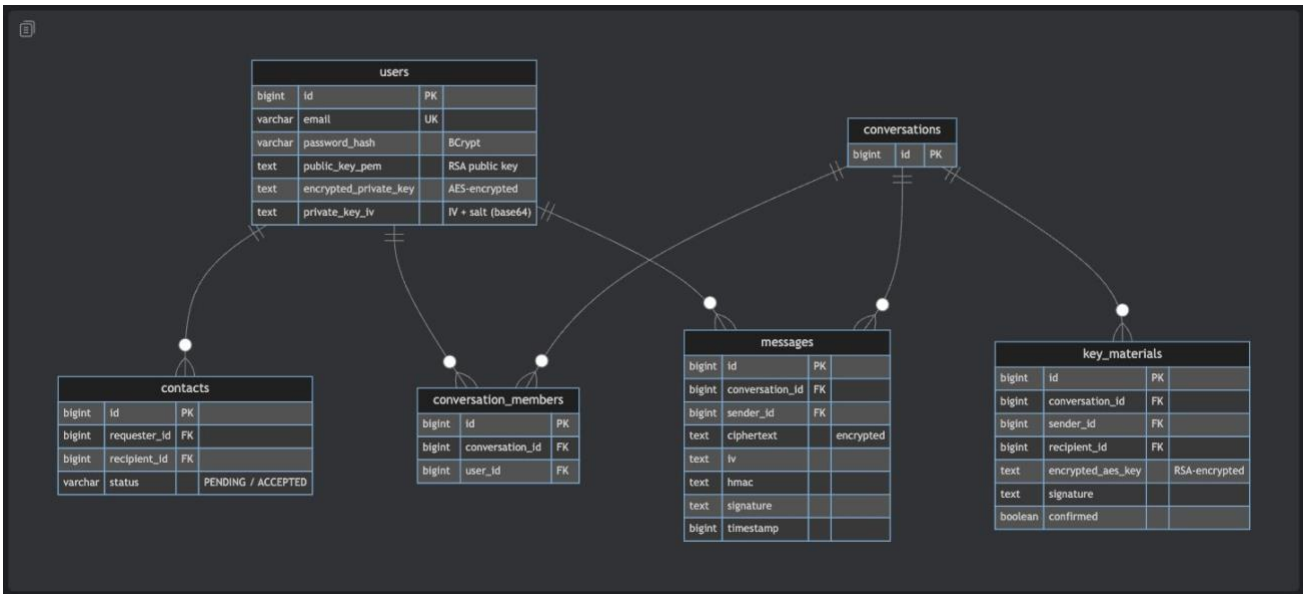

Figure 5 — Full Entity-Relationship diagram: users, contacts, conversations, conversation_members, messages, key_materials

7.1 Table Descriptions

Table	Contents & Security Notes

users	Email, BCrypt password hash, RSA public key (plaintext), AES-encrypted private key + IV/salt
contacts	Requester/recipient IDs + status (PENDING / ACCEPTED) — metadata only
conversations	Conversation ID only — no content
conversation_members	Links users to conversations by ID
messages	Ciphertext, IV, HMAC-SHA256, RSA-PSS signature, timestamp — all encrypted or hashed
key_materials	RSA-encrypted AES conversation keys + RSA-PSS signatures + confirmation flag

---

8. Security Analysis

8.1 Threat Model & Defenses

Threat	Defense
Network eavesdropper	TLS in transit; messages already AES-encrypted before leaving the browser
Database breach	All sensitive data is encrypted; no plaintext stored
Message forgery / impersonation	RSA-PSS signatures verified by server and recipient
Message tampering	HMAC-SHA256 over ciphertext + IV + timestamp
Replay attack (resend old message)	Server rejects timestamps outside ±5 min window
Timing side-channel on HMAC	constantTimeEquals() — XOR loop, no short-circuit
Password cracking after DB breach	BCrypt cost 12 — slow, salted, adaptive
Private key theft from server	Encrypted with PBKDF2 (600K iter) + AES-256-GCM; needs user password
Cross-conversation key compromise	Unique AES-256 key per conversation

8.2 What a Full Database Breach Exposes

Attacker Obtains	Can They Read Messages?
messages.ciphertext (encrypted blobs)	No — needs the conversation AES key

users.encrypted_private_key	No — needs password-derived key (PBKDF2 600K iterations)
users.password_hash	Extremely hard — BCrypt cost 12
contacts, conversation_members	Yes — metadata (who talks to whom) is exposed

8.3 Known Security Limitations

Limitation	Impact
No forward secrecy	Compromised conversation key exposes all messages in that conversation
Metadata visible to server	Server knows who messages whom, when, and message sizes
1:1 only — no group chat	Architecture supports only two-party conversations
No key rotation	Conversation keys never change
Single device only	Private key exists in only one browser session at a time

---

9. Testing

Test Suite	Coverage & Approach
Backend — 73 tests	Unit tests with Mockito for all services; MockMvc tests for all controllers; crypto round-trip tests using real RSA keys for signature verification
Frontend — 51 tests	Web Crypto round-trip tests for all four crypto modules; API layer tests with mocked fetch; React component tests with Testing Library; hook tests
End-to-End (Playwright)	Full simulation of two users registering, becoming contacts, and exchanging encrypted messages; verifies round-trip decryption

---

10. Conclusion

SecureChat proves that end-to-end encryption is achievable in a standard web application using only browser-native cryptography. The server is reduced to an encrypted blob store with signature verification — it cannot read messages even if fully compromised.

The combination of AES-256-GCM for encryption, RSA-PSS for authentication, HMAC-SHA256 for integrity, and timestamp validation for replay protection provides layered defense against a range of realistic threats. The hybrid RSA+AES architecture mirrors production systems such as WhatsApp and Signal.

The main acknowledged trade-offs are metadata exposure (the server knows who communicates with whom) and lack of forward secrecy (no key rotation). These limitations would require the Signal Double Ratchet protocol and additional metadata-protection techniques to address.